EXPLORA CANDELARIA

Manual técnico del sistema

Sistema de reservas turísticas · MySQL

Edición corregida 2.0 · 12 de octubre de 2026

Estructura adaptada del Manual Técnico CAE. Los recuadros indican las capturas y diagramas pendientes.

Índice

Introducción

Objetivo general

Objetivos específicos

1. Requerimientos de hardware y software

1.1. Hardware base del sistema

1.2. Software base del sistema

2. Instalación y configuración

2.1. Preparación del entorno

2.2. Configuración de PHP

2.3. Configuración de Apache o Nginx

2.4. Configuración de MySQL

2.5. Instalación de la aplicación

2.6. Conexión Laravel–MySQL

2.7. Variables de configuración

2.8. Arranque, tareas programadas y colas

2.9. Respaldo y restauración de MySQL

2.10. Logs y mantenimiento

3. Diseño del sistema

3.1. Arquitectura de la aplicación

3.2. Diagrama de flujo de datos

3.3. Arquitectura de red

3.4. Diagrama entidad–relación

3.5. Modelo relacional

3.6. Dependencias funcionales

3.7. Normalización

3.8. Diccionario de datos

actividades_festividades

actividades_turisticas

cache

cache_locks

cuentas_bancarias

detalle_alimentaciones

detalle_guianzas

detalle_hospedajes

detalle_paquete_turistico

emprendimiento_tipo_servicios

emprendimientos

failed_jobs

festividades

imagen_publicaciones

imagen_servicios

job_batches

jobs

migrations

model_has_permissions

model_has_roles

password_reset_tokens

permissions

publicaciones_turisticas

reserva_detalles

reservas

role_has_permissions

roles

servicios

sessions

sitios_turisticos

tipo_publicaciones

tipo_servicios

users

3.9. Casos de uso por rol

3.10. Diagramas de secuencia

3.11. Interfaz del sistema

4. Implementación

4.1. Módulos y reglas de negocio

4.2. Descripción de funciones y pseudocódigo

4.3. Despliegue y actualización

4.4. Pruebas y simulación

4.5. Limitaciones y diagnóstico

Anexo. Control de cambios y guía de capturas

Introducción

El sistema Explora Candelaria permite administrar información turística, emprendimientos, servicios y reservas de La Candelaria. Este manual técnico describe su arquitectura, instalación, configuración, diseño de datos y principales funciones para facilitar el mantenimiento y soporte.

Esta edición corrige el Word suministrado por el usuario y organiza sus contenidos siguiendo el ejemplo Manual Técnico CAE.pdf. El gestor de base de datos utilizado en el proyecto del usuario es MySQL. Las funciones se documentan con referencia a Rama-Daniela; la ubicación de los documentos en main no incorpora el código funcional de esa rama.

Fecha de edición: 12 de octubre de 2026. No se accedió al servidor MySQL del usuario. Su versión, nombre real de la base, tipos finales, índices, capacidad e instalación deben corroborarse mediante las capturas indicadas. Las comprobaciones locales anteriores no se presentan como validación de MySQL.

Objetivo general

Proporcionar una guía técnica para instalar, configurar, comprender y mantener el sistema de reservas turísticas Explora Candelaria con base de datos MySQL, apoyando al personal de desarrollo, infraestructura y soporte.

Objetivos específicos

Identificar requisitos de hardware y software; describir instalación y configuración; documentar arquitectura, relaciones y procesos; orientar respaldos y restauración; definir evidencias mediante capturas y registrar las limitaciones conocidas de la versión.

1. Requerimientos de hardware y software

1.1. Hardware base del sistema

Componente	Requerimiento / criterio
Servidor de aplicación	Equipo o máquina virtual con recursos para PHP, servidor web y procesos de fondo. Dimensionar según usuarios concurrentes y pruebas.
Servidor MySQL	Puede compartir el equipo en desarrollo o estar separado. Reservar memoria y almacenamiento para datos, índices y respaldos.
Equipo cliente	Navegador actualizado, conectividad y pantalla suficiente para los formularios.
Almacenamiento	Espacio para código, dependencias, base, imágenes, comprobantes, logs y copias de seguridad.
Datos pendientes	Registrar CPU, RAM, disco, sistema operativo y ubicación real. No se midió ni certificó un mínimo de producción.

CAPTURA 1 POR INSERTAR — Características del equipo o servidor
Insertar la pantalla de información del equipo o máquina virtual mostrando procesador, memoria, sistema operativo y almacenamiento. Ocultar identificadores privados.
Pie sugerido: Figura 1. Características del equipo o servidor.

1.2. Software base del sistema

Se recomienda PHP 8.4 y Composer 2 para el desarrollo. composer.json declara PHP ^8.2, pero el conjunto bloqueado de desarrollo contiene dependencias que requieren PHP 8.3 o superior. CI usa PHP 8.4 y 8.5. Confirma siempre los requisitos con composer check-platform-reqs.

Node 22 es la referencia de CI; Node 24 estaba disponible en el entorno usado para elaborar los documentos. Vite 7 necesita una versión de Node compatible. Instala con npm ci para conservar el lockfile.

Extensiones: PDO y pdo_mysql para la base MySQL; DOM/XML, mbstring, OpenSSL, Fileinfo, GD, ZIP y las demás indicadas por Composer. La compatibilidad debe comprobarse tanto en PHP CLI como en el servidor web.

Necesitas escritura en storage y bootstrap/cache, espacio para dependencias y archivos subidos, y acceso a los repositorios y servicios externos utilizados. No se establece una capacidad de producción garantizada: dimensiona con carga representativa y monitoreo.

```
php -v
php -m
composer --version
node --version
npm --version
composer check-platform-reqs
```

Software	Referencia
MySQL	Motor confirmado por el usuario. Anotar versión real con SELECT VERSION(); se recomienda una versión compatible vigente, por ejemplo MySQL 8.x.
Motor de tablas	InnoDB para tablas relacionadas y operaciones transaccionales; verificar en la instalación real.
Codificación	utf8mb4; confirmar cotejamiento del esquema y compatibilidad con Laravel.
Servidor HTTP	Apache o Nginx con PHP-FPM según el entorno. No se confirmó cuál usa el usuario.
Herramienta de administración	MySQL Workbench, phpMyAdmin o cliente mysql; opcional según la instalación.

CAPTURA 2 POR INSERTAR — Versiones de PHP, Composer, Node y MySQL
Mostrar php -v, composer --version, node --version y SELECT VERSION() de la instalación real. No mostrar credenciales.
Pie sugerido: Figura 2. Versiones de PHP, Composer, Node y MySQL.

2. Instalación y configuración

2.1. Preparación del entorno

Trabaja sobre un checkout de la rama funcional que deseas ejecutar. Conserva cambios locales. Instala PHP, Composer, Node, MySQL y el servidor HTTP con el gestor oficial del sistema operativo. El PDF de ejemplo usa tecnologías antiguas distintas; no deben trasladarse sus comandos PostgreSQL, versiones de PHP ni herramientas obligatorias a este proyecto Laravel.

2.2. Configuración de PHP

Habilita pdo_mysql y las extensiones exigidas por Composer, incluyendo DOM/XML, mbstring, OpenSSL, Fileinfo, GD y ZIP. Identifica el php.ini cargado por CLI y por el servidor web; pueden ser diferentes. Configura límites de carga suficientes para las validaciones de archivos y display_errors desactivado en producción. La zona horaria debe ser coherente entre aplicación y servidor.

```
php --ini
php -m
composer check-platform-reqs
```

CAPTURA 3 POR INSERTAR — Extensiones PHP y archivo php.ini
Mostrar pdo_mysql en php -m y la ubicación de php.ini. Como complemento, insertar configuración de límites de carga y zona horaria; ocultar rutas privadas si corresponde.
Pie sugerido: Figura 3. Extensiones PHP y archivo php.ini.

2.3. Configuración de Apache o Nginx

La raíz pública del sitio debe apuntar a public, nunca al directorio completo del proyecto. Configura la resolución de rutas Laravel, PHP-FPM o el módulo PHP apropiado, HTTPS en producción y permisos de escritura en storage y bootstrap/cache. Utiliza el archivo de configuración correspondiente al servidor realmente instalado.

CAPTURA 4 POR INSERTAR — Configuración del servidor web
Mostrar el VirtualHost o server block con raíz apuntando a la carpeta public, nombre del sitio y tratamiento de PHP. No incluir claves TLS privadas.
Pie sugerido: Figura 4. Configuración del servidor web.

2.4. Configuración de MySQL

Verifica que MySQL esté activo y accesible desde la aplicación. No expongas el puerto de la base a Internet salvo una necesidad justificada y controles de acceso. Crea un esquema con utf8mb4 y un usuario limitado al esquema. Usa cuentas diferenciadas para aplicar migraciones y para la operación cuando el despliegue lo permita.

```
mysql -u root -p
# Dentro del cliente MySQL:
SELECT VERSION();
CREATE DATABASE reservas_turisticas
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

reservas_turisticas es un nombre de ejemplo: utiliza el nombre real. Crea la cuenta y su contraseña mediante el procedimiento seguro de tu instalación. La cuenta de migraciones necesita permisos para crear y alterar objetos; la de operación debe recibir solo los permisos necesarios. Evita utilizar root como usuario de la aplicación.

CAPTURA 5 POR INSERTAR — Base de datos MySQL creada
Mostrar en Workbench o phpMyAdmin el nombre del esquema y su codificación/cotejamiento. Utilizar el nombre real del proyecto.
Pie sugerido: Figura 5. Base de datos MySQL creada.

CAPTURA 6 POR INSERTAR — Usuario y privilegios del esquema
Mostrar los privilegios asociados al esquema, sin contraseña ni información de otras bases.
Pie sugerido: Figura 6. Usuario y privilegios del esquema.

2.5. Instalación de la aplicación

Utiliza el checkout existente cuando ya esté preparado. Para una instalación nueva de la versión documentada, clona Rama-Daniela. Conserva los cambios locales y revisa el estado de Git antes de instalar. Los siguientes comandos son instrucciones de instalación, no una publicación automática.

```
git clone --branch Rama-Daniela --single-branch \
  https://github.com/jhinier/proyecto_reservas_turisticas.git
cd proyecto_reservas_turisticas
composer install --no-interaction --prefer-dist
npm ci
```

Crea .env a partir de .env.example únicamente si no existe. Configura la base, URL y correo. En una instalación inicial sin clave, genera APP_KEY; nunca regeneres una clave de una instalación con datos cifrados existentes.

```
php -r "file_exists('.env') || copy('.env.example', '.env');"
# Solo en una instalación nueva sin APP_KEY:
php artisan key:generate
# Crea el esquema MySQL y configura .env antes de migrar.
php artisan migrate
php artisan storage:link
npm run build
php artisan migrate:status
php artisan route:list
```

Ejecuta las migraciones antes de cargar catálogos. TipoServicioSeeder contiene Guianza, Paquetes Turísticos, Alimentación, Hospedaje y Alquiler de Equipos. Existe TipoPublicacionSeeder para el catálogo de publicaciones. Revisa los seeders antes de ejecutarlos.

```
php artisan db:seed --class=TipoServicioSeeder
php artisan db:seed --class=TipoPublicacionSeeder
```

DatabaseSeeder llama a permisos, que crea roles, permisos y una cuenta administrativa con datos prefijados. No ejecutes este seeder indiscriminadamente en una base existente o producción. Revisa la cuenta inicial, establece credenciales privadas y evita repetir operaciones no idempotentes. Ninguna contraseña del seeder se reproduce en este manual.

Si vendor está versionado en un checkout distinto, una instalación normal puede modificar archivos protegidos. Comprueba git ls-files vendor y utiliza una copia aislada o un procedimiento aprobado de actualización. La rama funcional inspeccionada no versiona vendor. No cambies lockfiles para resolver una instalación sin investigar antes el requisito ausente.

CAPTURA 7 POR INSERTAR — Instalación de dependencias
Mostrar el resultado correcto de composer install y npm ci, desde la carpeta de la rama funcional.
Pie sugerido: Figura 7. Instalación de dependencias.

2.6. Conexión Laravel–MySQL

Configura .env con el motor mysql, el host, puerto, esquema y usuario reales. El bloque siguiente es una plantilla, no una configuración comprobada del servidor del usuario. Introduce la contraseña únicamente en el archivo local o gestor seguro; no la publiques en el documento ni en Git.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=reservas_turisticas
DB_USERNAME=usuario_aplicacion
DB_PASSWORD=<configurar_en_el_entorno>
```

Después de cambiar .env, limpia la configuración cacheada cuando corresponda. Prueba la conexión contra la base de desarrollo correcta antes de aplicar migraciones. El ejemplo 127.0.0.1 solo sirve si MySQL reside en el mismo entorno; sustituye el host si está en otro servidor.

```
php artisan config:clear
php artisan migrate:status
# En una base nueva preparada y con la cuenta autorizada:
php artisan migrate
```

CAPTURA 8 POR INSERTAR — Configuración de conexión MySQL en .env
Mostrar DB_CONNECTION=mysql, host, puerto, nombre de base y usuario. Sustituir la contraseña por asteriscos; ocultar APP_KEY y otras credenciales.
Pie sugerido: Figura 8. Configuración de conexión MySQL en .env.

CAPTURA 9 POR INSERTAR — Migraciones y tablas en MySQL
Mostrar php artisan migrate:status y el listado de tablas de la base real. Confirmar que se trata de MySQL y del esquema del proyecto.
Pie sugerido: Figura 9. Migraciones y tablas en MySQL.

2.7. Variables de configuración

Variables	Uso
APP_NAME, APP_ENV, APP_URL	Nombre, entorno y URL pública; enlaces de correos dependen de APP_URL.
APP_KEY	Clave de cifrado; conservarla y respaldarla de forma segura.
APP_DEBUG	true en desarrollo; false en producción.
DB_CONNECTION, DB_DATABASE	Motor y base. En MySQL, DB_DATABASE es el nombre del esquema, no una ruta de archivo.
DB_HOST, DB_PORT, DB_USERNAME, DB_PASSWORD	Conexión a motores remotos cuando corresponda.
MAIL_MAILER, MAIL_HOST, MAIL_PORT	Transporte y servidor de correo.
MAIL_USERNAME, MAIL_PASSWORD	Autenticación del correo; valores privados fuera de Git.
MAIL_FROM_ADDRESS, MAIL_FROM_NAME	Remitente de los mensajes.
QUEUE_CONNECTION	Backend de colas; .env.example propone database.
CACHE_STORE, SESSION_DRIVER	Backend de caché y sesiones; ejemplo con database.

Variables	Uso
FILESYSTEM_DISK	Disco de almacenamiento predeterminado.
LOG_CHANNEL, LOG_LEVEL	Canal y nivel de registros.
AWS_*, REDIS_*	Solo si el despliegue utiliza esos proveedores.

Para desarrollo local, MAIL_MAILER=log registra el contenido de los mensajes y no entrega correo real. No compartas registros que contengan enlaces de verificación, datos personales o credenciales. La confirmación del registro pendiente se conserva en caché durante 30 minutos: el backend debe mantenerse disponible durante ese periodo.

No subas .env, auth.json, credenciales, claves privadas o bases de datos con información real. Los requisitos FLUX_USERNAME y FLUX_LICENSE_KEY aparecen en CI; su ausencia no demuestra por sí sola que los paquetes utilizados requieran licencia privada. Comprueba la operación real y solicita credenciales solo para repositorios privados que efectivamente las necesiten.

2.8. Arranque, tareas programadas y colas

En desarrollo convencional, composer dev inicia servidor, escucha de colas y Vite. El programador de tareas requiere un proceso adicional. También puedes iniciar cada proceso en una terminal independiente para identificar fallos.

```
php artisan serve
php artisan queue:listen --tries=1
npm run dev
php artisan schedule:work
```

En el entorno de nube preparado, PHP está instalado dentro de /workspace/.cloud-tools. Activa su envoltura en cada shell. Laravel debe conservar las variables del PHP local al crear el servidor; por eso se usa --no-reload. Este ajuste es específico de ese entorno.

```
export PATH=/workspace/.cloud-tools/bin:$PATH
export COMPOSER_HOME=/workspace/.cloud-tools/composer-home
cd /workspace/manual-rama-daniela
php artisan serve --no-reload --host=127.0.0.1 --port=8010
```

routes/console.php registra procesarCancelacionesAutomaticas cada minuto. En producción, ejecuta schedule:run desde cron cada minuto y supervisa los trabajadores de cola. Evita iniciar dos programadores para la misma instancia sin definir cómo impedir ejecución duplicada.

```
* * * * * cd /ruta/al/proyecto && php artisan schedule:run >> /ruta/al/log/scheduler.
log 2>&1
```

No uses artisan serve ni el servidor de Vite como servidores de producción. Publica los recursos compilados, configura un servidor HTTP con PHP-FPM y usa public como raíz web. Muchos mensajes de reserva llaman Mail::send directamente; disponer de un worker no convierte esos envíos en asíncronos.

CAPTURA 10 POR INSERTAR — Aplicación en ejecución
Mostrar la página inicial con URL del entorno autorizado y una consola de arranque si corresponde. Evitar exponer direcciones internas privadas.
Pie sugerido: Figura 10. Aplicación en ejecución.

CAPTURA 11 POR INSERTAR — Programador de tareas y colas
Mostrar php artisan schedule:list, el proceso de colas y la configuración cron o equivalente. No incluir contenido privado de mensajes.
Pie sugerido: Figura 11. Programador de tareas y colas.

2.9. Respaldo y restauración de MySQL

Respalda la base MySQL, archivos de storage/app, configuración privada y APP_KEY, junto con el commit y los lockfiles. Para tablas InnoDB, mysqldump con --single-transaction facilita una copia consistente sin bloquear transacciones ordinarias; evita cambios de esquema durante el respaldo. Usa una cuenta con los permisos exigidos por la versión y los objetos que incluyas.

```
mysqldump -h 127.0.0.1 -u usuario_backup -p \
  --single-transaction --no-tablespaces \
  reservas_turisticas > respaldo_reservas.sql
# Restauración SOLO en un esquema de prueba creado previamente:
mysql -h 127.0.0.1 -u usuario_restore -p \
  reservas_turisticas_prueba < respaldo_reservas.sql
```

Los nombres son ejemplos. Si hay rutinas, eventos o triggers, inclúyelos conforme al alcance y permisos del respaldo. Nunca escribas la contraseña en la línea de comandos. Verifica el resultado y prueba restauraciones en un esquema aislado; una instrucción documentada no prueba que la recuperación haya sido ejecutada.

CAPTURA 12 POR INSERTAR — Respaldo MySQL
Mostrar la herramienta o consola de respaldo con ejecución satisfactoria y nombre del archivo; ocultar usuarios privados y rutas sensibles.
Pie sugerido: Figura 12. Respaldo MySQL.

CAPTURA 13 POR INSERTAR — Restauración en esquema de prueba
Mostrar el esquema de prueba restaurado y una consulta de comprobación. No usar capturas de datos personales reales.
Pie sugerido: Figura 13. Restauración en esquema de prueba.

2.10. Logs y mantenimiento

Respalda la base, storage/app con archivos cargados, configuración privada y APP_KEY mediante un canal seguro; conserva además el commit y lockfiles de la versión. Los documentos del repositorio no sustituyen un respaldo de datos. Las credenciales y respaldos personales no se almacenan en GitHub.

Para MySQL utiliza respaldos consistentes, conserva los archivos cargados y define retención. Prueba periódicamente la restauración en un esquema aislado y con permisos controlados.

Orden recomendado de recuperación: preparar runtime y código compatible; restaurar base y archivos; recuperar la clave y configuración; comprobar permisos; iniciar servicios; verificar autenticación, archivos, reservas y tareas. No pruebes la restauración sobre la única copia de producción.

Controla logs, fallos de correo, trabajos fallidos, estado del cron, espacio en disco y errores de consultas. Si cambias tablas o reglas de reserva, actualiza este manual y el manual de usuario. Revisa enlaces públicos de archivos, límites de carga, roles y políticas de acceso.

```
php artisan queue:failed
php artisan schedule:list
php artisan migrate:status
php artisan optimize:clear
```

CAPTURA 14 POR INSERTAR — Registro de errores del sistema
Mostrar una entrada sanitizada de storage/logs/laravel.log con fecha, nivel y mensaje. Ocultar correos, tokens, datos bancarios y contraseñas.
Pie sugerido: Figura 14. Registro de errores del sistema.

3. Diseño del sistema

3.1. Arquitectura de la aplicación

La aplicación utiliza Laravel como servidor, Livewire para componentes interactivos, Blade para vistas y Eloquent para persistencia. Las rutas aplican autenticación y roles; controladores y componentes delegan operaciones a servicios de negocio. Vite construye los recursos estáticos. El navegador también carga recursos externos del mapa y de las vistas públicas.

Componente	Versión del lock / función
Laravel	12.53.0 · aplicación, rutas y persistencia
Livewire	4.2.1 · interacción de componentes
Flux	2.13.0 · componentes de interfaz
Fortify	1.35.0 · autenticación y recuperación
Spatie Permission	7.2.2 · roles y permisos
Dompdf	3.1.2 · reportes PDF
PhpSpreadsheet	5.7.0 · dependencia declarada; no implica que todos los reportes exporten Excel
Pest	4.4.1 · pruebas
Vite / Tailwind	Vite 7 y Tailwind 4 según package.json
Otras dependencias	FullCalendar 6, Chart.js 4, Axios y google/cloud-translate

Flujo de una solicitud: navegador → rutas y middleware → controlador o componente Livewire → servicio → modelo Eloquent → base de datos. Las vistas Blade presentan el resultado. Los archivos se guardan mediante Storage y los correos mediante Mail.

Capa	Responsabilidad
Presentación	resources/views, layouts de turista y paneles, componentes Livewire.
Aplicación	routes, controladores, acciones Fortify y proveedores.
Negocio	app/Services: reservas, inventario, servicios y emprendimientos.
Datos	app/Models, database/migrations y seeders.
Operación	config, storage, colas y routes/console.php.

Directorio / archivo	Contenido
app/Livewire/Admin	Gestión administrativa y publicaciones.
app/Livewire/Emprendimiento	Servicios, reservas y reportes del establecimiento.

Directorio / archivo	Contenido
app/Livewire/Turista	Buscador, mapa, checkout e historial.
app/Livewire/Settings	Perfil, contraseña, apariencia, 2FA, datos comerciales y de pago.
app/Services	Operaciones y reglas de negocio.
app/Mail y app/Notifications	Mensajes de registro, recuperación y reservas.
app/Simulacion	Simulador de rendimiento y generación de reportes de simulación.
routes/web.php / settings.php	Rutas funcionales y configuración personal.
routes/console.php	Tarea de revisión de reservas cada minuto.
bootstrap/app.php	Configuración inicial y alias de middleware.
database/migrations	Evolución del esquema; fuente de referencia de la base.
resources / public	Código de recursos, vistas y archivos accesibles por HTTP.
storage	Archivos, registros, sesiones y cachés locales.
composer.lock / package-lock.json	Versiones resueltas de dependencias.
.github/workflows	Comprobaciones de formato y pruebas en CI.

CAPTURA 15 POR INSERTAR — Diagrama de arquitectura de la aplicación
Insertar un diagrama: navegador → rutas/middleware → controladores y Livewire → servicios → modelos Eloquent → MySQL. Añadir almacenamiento, correo y programador como componentes externos de operación.
Pie sugerido: Figura 15. Diagrama de arquitectura de la aplicación.

3.2. Diagrama de flujo de datos

Nivel de contexto: turistas, administradores y emprendedores intercambian datos con Explora Candelaria. El sistema consulta y persiste información en MySQL y entrega notificaciones por correo. En el nivel de procesos, separar autenticación, publicaciones, catálogo, reservas/pagos y reportes. Identificar entradas, salidas y almacenes sin confundir el diagrama de datos con una secuencia de llamadas.

CAPTURA 16 POR INSERTAR — DFD de contexto
Mostrar los tres actores, el proceso Explora Candelaria y los flujos de datos: registro, consulta, gestión de oferta, reserva, revisión de pago y reportes.
Pie sugerido: Figura 16. DFD de contexto.

CAPTURA 17 POR INSERTAR — DFD de nivel 1
Descomponer autenticación, catálogo/publicaciones, reservas/pagos y reportes; conectar los almacenes usuarios, servicios, publicaciones y reservas de MySQL.
Pie sugerido: Figura 17. DFD de nivel 1.

3.3. Arquitectura de red

El cliente accede por HTTPS al servidor web y PHP. Laravel se conecta al host MySQL configurado. Correo y recursos de mapas/CDN son destinos externos. Puede existir un servidor único en desarrollo; el diagrama debe representar la instalación real, no una separación de servidores supuesta.

CAPTURA 18 POR INSERTAR — Diagrama de red del despliegue real
Insertar navegador → servidor web/PHP → MySQL, con puertos y red solo si son confirmados. Añadir SMTP y servicios externos. Usar nombres genéricos en lugar de IP privadas sensibles.
Pie sugerido: Figura 18. Diagrama de red del despliegue real.

3.4. Diagrama entidad–relación

El esquema de referencia de las migraciones contiene 33 tablas, incluidas las técnicas de Laravel; confirmar el total real en MySQL. Las fechas de los servicios reservados viven en reserva_detalles. La relación del establecimiento con la reserva se obtiene a través de los servicios y su categoría pivote.

Entidad	Responsabilidad y relación
users	Datos personales, acceso y verificación; se asocia a reservas y emprendimientos.
emprendimientos	Establecimiento y responsable user_id. Tiene categorías y cuentas bancarias.
tipo_servicios	Catálogo de categorías.
emprendimiento_tipo_servicios	Pivote establecimiento/categoría con estado.
servicios	Oferta; pertenece al pivote; precio y stock.
detalle_hospedajes / detalle_guianzas	Propiedades específicas: capacidad o personas.
detalle_alimentaciones / detalle_paquete_turistico	Datos de alimentación o del paquete.
imagen_servicios	Imágenes asociadas a un servicio.
reservas	Cabecera: turista, estado, total, auditoría y comprobante.
reserva_detalles	Líneas: servicio, fechas, hora, cantidad, personas, precio unitario y subtotal.
cuentas_bancarias	Banco, número y titular del emprendimiento.
publicaciones_turisticas	Cabecera de contenido turístico asociada a usuario y tipo.
sitios_turisticos / actividades_turisticas / festividades	Extensiones de una publicación.
actividades_festividades	Programación de una festividad.
imagen_publicaciones	Galería de publicaciones.
roles / permissions y pivotes	Autorización mediante Spatie.
sessions / cache / jobs y auxiliares	Infraestructura Laravel.

Relaciones principales: User 1:N Reserva; Reserva 1:N ReservaDetalle; Servicio 1:N ReservaDetalle; Emprendimiento 1:N EmprendimientoTipoServicio; TipoServicio 1:N EmprendimientoTipoServicio; EmprendimientoTipoServicio 1:N Servicio; Emprendimiento 1:N CuentaBancaria. Las cardinalidades describen las claves y modelos; verifica restricciones únicas antes de asumir una relación exclusiva de negocio.

users, emprendimientos y servicios utilizan borrado lógico. Las claves de varias tablas incluyen eliminación en cascada para borrado físico. Evalúa las consecuencias antes de purgar registros: borrar lógicamente, borrar físicamente y cancelar una reserva son operaciones distintas.

CAPTURA 19 POR INSERTAR — Diagrama entidad–relación en MySQL
Obtener o elaborar el ER desde el esquema real en Workbench. Mostrar users, emprendimientos, tipos/pivote/servicios, reservas/detalles, cuentas y publicaciones; señalar PK, FK y cardinalidades. Dividir en varias imágenes si no es legible.
Pie sugerido: Figura 19. Diagrama entidad–relación en MySQL.

3.5. Modelo relacional

Relaciones centrales: users(id) → reservas(user_id); users(id) → emprendimientos(user_id); emprendimientos y tipo_servicios → emprendimiento_tipo_servicios; pivote → servicios; reservas y servicios → reserva_detalles; emprendimientos → cuentas_bancarias. Las publicaciones tienen tablas de tipo, detalle y galería. El diccionario siguiente conserva los nombres de los objetos del proyecto y requiere contraste con MySQL.

CAPTURA 20 POR INSERTAR — Modelo relacional y claves foráneas
Mostrar las claves de reservas, reserva_detalles, servicios y emprendimiento_tipo_servicios desde Workbench o SHOW CREATE TABLE; incluir nombres y referencias de las FK.
Pie sugerido: Figura 20. Modelo relacional y claves foráneas.

3.6. Dependencias funcionales

Las PK identifican las filas y determinan sus atributos. Ejemplos: servicios.id determina categoría pivote, nombre, descripción, precio y stock; reservas.id determina turista, estado y total; reserva_detalles.id determina reserva, servicio, fechas y valores de la línea. No afirmar que una combinación de columnas es única salvo que la restricción exista en MySQL.

precio_unitario y subtotal en reserva_detalles guardan un valor histórico: no deben sustituirse automáticamente por el precio actual de servicios. La redundancia del total de cabecera responde a una decisión de negocio y debe verificarse cuando se modifican líneas.

3.7. Normalización

El diseño separa usuarios, establecimientos, categorías, servicios y detalles para evitar repetición de información. El pivote representa la relación establecimiento–categoría; la cabecera y líneas representan una reserva con varios servicios. Esto permite explicar las formas normales, pero no constituye una auditoría formal de 3FN o BCNF.

Verifica 1FN según atomicidad de cada atributo; 2FN según dependencias de claves compuestas; 3FN según dependencias transitivas. Los campos JSON, enlaces múltiples y valores derivados requieren revisar requisitos concretos. Documenta toda desnormalización deliberada y sus mecanismos de consistencia.

3.8. Diccionario de datos

El diccionario del Word original fue generado desde un esquema local de prueba y NO es una extracción del MySQL del usuario. Se conservan los nombres de tablas y columnas para orientar la revisión, pero los tipos se presentan como correspondencias orientativas. Contrasta cada tabla con SHOW CREATE TABLE e INFORMATION_SCHEMA; las migraciones son la fuente del esquema esperado. No se certifican tipos, longitudes, índices ni nulabilidad del servidor real.

```
SHOW TABLES;
SHOW CREATE TABLE reservas;
SHOW CREATE TABLE reserva_detalles;
SELECT TABLE_NAME, COLUMN_NAME, COLUMN_TYPE, IS_NULLABLE,
       COLUMN_KEY, COLUMN_DEFAULT
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = DATABASE()
ORDER BY TABLE_NAME, ORDINAL_POSITION;
```

CAPTURA 21 POR INSERTAR — Diccionario real de MySQL
Mostrar la estructura de una tabla central con nombre, tipo, tamaño, nulabilidad, PK y FK. Exportar las estructuras de las demás tablas para contrastar el diccionario completo.
Pie sugerido: Figura 21. Diccionario real de MySQL.

Correspondencia orientativa de tablas y columnas del documento original. Confirmar tipos, tamaños, PK, FK y nulabilidad mediante SHOW CREATE TABLE en MySQL; las marcas heredadas no certifican restricciones reales.

actividades_festividades

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
publicacion_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
nombre	varchar	NOT NULL; confirmar en MySQL
fecha	date	NOT NULL; confirmar en MySQL
hora	time	NOT NULL; confirmar en MySQL
lugar	varchar	NOT NULL; confirmar en MySQL
descripcion	varchar	NOT NULL; confirmar en MySQL
imagen	varchar	según PK / admite NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: publicacion_id → publicaciones_turisticas.id; ON DELETE CASCADE

actividades_turisticas

Columna	Tipo orientativo para contrastar	Restricción por verificar
publicacion_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
duracion_estimada	varchar	NOT NULL; confirmar en MySQL
dificultad	varchar	NOT NULL; confirmar en MySQL
recomendaciones	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: publicacion_id → publicaciones_turisticas.id; ON DELETE CASCADE

cache

Columna	Tipo orientativo para contrastar	Restricción por verificar
key	varchar	PK NOT NULL; confirmar en MySQL
value	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
expiration	BIGINT o INT según migración	NOT NULL; confirmar en MySQL

cache_locks

Columna	Tipo orientativo para contrastar	Restricción por verificar
key	varchar	PK NOT NULL; confirmar en MySQL
owner	varchar	NOT NULL; confirmar en MySQL
expiration	BIGINT o INT según migración	NOT NULL; confirmar en MySQL

cuentas_bancarias

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
emprendimiento_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
nombre_banco	varchar	NOT NULL; confirmar en MySQL
numero_cuenta	varchar	NOT NULL; confirmar en MySQL
titular	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL

Columna	Tipo orientativo para contrastar	Restricción por verificar
updated_at	datetime	según PK / admite NULL; confirmar en MySQL
deleted_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: emprendimiento_id → emprendimientos.id; ON DELETE CASCADE

detalle_alimentaciones

Columna	Tipo orientativo para contrastar	Restricción por verificar
servicio_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
tipo_alimentacion	varchar	NOT NULL; confirmar en MySQL
lugar_alimentacion	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: servicio_id → servicios.id; ON DELETE CASCADE

detalle_guianzas

Columna	Tipo orientativo para contrastar	Restricción por verificar
servicio_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
numero_max_persona	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: servicio_id → servicios.id; ON DELETE CASCADE

detalle_hospedajes

Columna	Tipo orientativo para contrastar	Restricción por verificar
servicio_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
capacidad	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL

Columna	Tipo orientativo para contrastar	Restricción por verificar
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: servicio_id → servicios.id; ON DELETE CASCADE

detalle_paquete_turistico

Columna	Tipo orientativo para contrastar	Restricción por verificar
servicio_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
lugar_salida	varchar	NOT NULL; confirmar en MySQL
hora_salida	time	NOT NULL; confirmar en MySQL
servicios_incluidos	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
lugares_actividades	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
recomendaciones	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
documento	varchar	NOT NULL; confirmar en MySQL
mensaje_pago	varchar	según PK / admite NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL
duracion_dias	BIGINT o INT según migración	NOT NULL; confirmar en MySQL

Claves foráneas: servicio_id → servicios.id; ON DELETE CASCADE

emprendimiento_tipo_servicios

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
tipo_servicio_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
emprendimiento_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL
estado	tinyint(1)	NOT NULL; confirmar en MySQL

Claves foráneas: emprendimiento_id → emprendimientos.id; ON DELETE CASCADE; tipo_servicio_id → tipo_servicios.id; ON DELETE CASCADE

emprendimientos

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
user_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
nombre	varchar	NOT NULL; confirmar en MySQL
descripcion	varchar	NOT NULL; confirmar en MySQL
estado	tinyint(1)	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL
imagen	varchar	según PK / admite NULL; confirmar en MySQL
deleted_at	datetime	según PK / admite NULL; confirmar en MySQL
enlaces	TEXT / VARCHAR según migración	según PK / admite NULL; confirmar en MySQL

Claves foráneas: user_id → users.id; ON DELETE CASCADE

failed_jobs

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
uuid	varchar	NOT NULL; confirmar en MySQL
connection	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
queue	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
payload	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
exception	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
failed_at	datetime	NOT NULL; confirmar en MySQL

festividades

Columna	Tipo orientativo para contrastar	Restricción por verificar
publicacion_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
fecha_inicio	date	NOT NULL; confirmar en MySQL
fecha_fin	date	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: publicacion_id → publicaciones_turisticas.id; ON DELETE CASCADE

imagen_publicaciones

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
publicacion_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
imagen	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: publicacion_id → publicaciones_turisticas.id; ON DELETE CASCADE

imagen_servicios

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
servicio_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
imagen	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: servicio_id → servicios.id; ON DELETE CASCADE

job_batches

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	varchar	PK NOT NULL; confirmar en MySQL
name	varchar	NOT NULL; confirmar en MySQL
total_jobs	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
pending_jobs	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
failed_jobs	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
failed_job_ids	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
options	TEXT / VARCHAR según migración	según PK / admite NULL; confirmar en MySQL
cancelled_at	BIGINT o INT según migración	según PK / admite NULL; confirmar en MySQL
created_at	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
finished_at	BIGINT o INT según migración	según PK / admite NULL; confirmar en MySQL

jobs

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
queue	varchar	NOT NULL; confirmar en MySQL
payload	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
attempts	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
reserved_at	BIGINT o INT según migración	según PK / admite NULL; confirmar en MySQL
available_at	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
created_at	BIGINT o INT según migración	NOT NULL; confirmar en MySQL

migrations

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
migration	varchar	NOT NULL; confirmar en MySQL
batch	BIGINT o INT según migración	NOT NULL; confirmar en MySQL

model_has_permissions

Columna	Tipo orientativo para contrastar	Restricción por verificar
permission_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
model_type	varchar	PK NOT NULL; confirmar en MySQL
model_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL

Claves foráneas: permission_id → permissions.id; ON DELETE CASCADE

model_has_roles

Columna	Tipo orientativo para contrastar	Restricción por verificar
role_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
model_type	varchar	PK NOT NULL; confirmar en MySQL
model_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL

Claves foráneas: role_id → roles.id; ON DELETE CASCADE

password_reset_tokens

Columna	Tipo orientativo para contrastar	Restricción por verificar
email	varchar	PK NOT NULL; confirmar en MySQL
token	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL

permissions

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
name	varchar	NOT NULL; confirmar en MySQL
guard_name	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

publicaciones_turisticas

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
user_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
tipo_publicacion_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
nombre	varchar	NOT NULL; confirmar en MySQL
descripcion	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: tipo_publicacion_id → tipo_publicaciones.id; ON DELETE CASCADE; user_id → users.id; ON DELETE CASCADE

reserva_detalles

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
reserva_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
servicio_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
fecha_inicio	date	NOT NULL; confirmar en MySQL
fecha_fin	date	según PK / admite NULL; confirmar en MySQL
cantidad	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
precio_unitario	numeric	NOT NULL; confirmar en MySQL
subtotal	numeric	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL
hora_llegada	time	según PK / admite NULL; confirmar en MySQL
numero_personas	BIGINT o INT según migración	según PK / admite NULL; confirmar en MySQL

Claves foráneas: reserva_id → reservas.id; ON DELETE CASCADE; servicio_id → servicios.id; ON DELETE CASCADE

reservas

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
user_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
estado	varchar	NOT NULL; confirmar en MySQL
precio_total	numeric	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL
reservada_por_rol	varchar	NOT NULL; confirmar en MySQL
cancelada_en	datetime	según PK / admite NULL; confirmar en MySQL
cancelada_por_rol	varchar	según PK / admite NULL; confirmar en MySQL
motivo_cancelacion	TEXT / VARCHAR según migración	según PK / admite NULL; confirmar en MySQL
comprobante_pago	varchar	según PK / admite NULL; confirmar en MySQL
fecha_subida_comprobante	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: user_id → users.id; ON DELETE CASCADE

role_has_permissions

Columna	Tipo orientativo para contrastar	Restricción por verificar
permission_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
role_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL

Claves foráneas: role_id → roles.id; ON DELETE CASCADE; permission_id → permissions.id; ON DELETE CASCADE

roles

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
name	varchar	NOT NULL; confirmar en MySQL
guard_name	varchar	NOT NULL; confirmar en MySQL

Columna	Tipo orientativo para contrastar	Restricción por verificar
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

servicios

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
emprendimiento_tipo_servicio_id	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
nombre	varchar	NOT NULL; confirmar en MySQL
descripcion	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
precio	numeric	NOT NULL; confirmar en MySQL
stock	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL
deleted_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: emprendimiento_tipo_servicio_id → emprendimiento_tipo_servicios.id; ON DELETE CASCADE

sessions

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	varchar	PK NOT NULL; confirmar en MySQL
user_id	BIGINT o INT según migración	según PK / admite NULL; confirmar en MySQL
ip_address	varchar	según PK / admite NULL; confirmar en MySQL
user_agent	TEXT / VARCHAR según migración	según PK / admite NULL; confirmar en MySQL
payload	TEXT / VARCHAR según migración	NOT NULL; confirmar en MySQL
last_activity	BIGINT o INT según migración	NOT NULL; confirmar en MySQL

sitios_turisticos

Columna	Tipo orientativo para contrastar	Restricción por verificar
publicacion_id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
ubicacion	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

Claves foráneas: publicacion_id → publicaciones_turisticas.id; ON DELETE CASCADE

tipo_publicaciones

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
nombre	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

tipo_servicios

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
nombre	varchar	NOT NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL

users

Columna	Tipo orientativo para contrastar	Restricción por verificar
id	BIGINT o INT según migración	PK NOT NULL; confirmar en MySQL
name	varchar	NOT NULL; confirmar en MySQL
apellidos	varchar	NOT NULL; confirmar en MySQL
email	varchar	NOT NULL; confirmar en MySQL

Columna	Tipo orientativo para contrastar	Restricción por verificar
cedula	varchar	NOT NULL; confirmar en MySQL
edad	BIGINT o INT según migración	NOT NULL; confirmar en MySQL
telefono	varchar	NOT NULL; confirmar en MySQL
email_verified_at	datetime	según PK / admite NULL; confirmar en MySQL
password	varchar	NOT NULL; confirmar en MySQL
remember_token	varchar	según PK / admite NULL; confirmar en MySQL
created_at	datetime	según PK / admite NULL; confirmar en MySQL
updated_at	datetime	según PK / admite NULL; confirmar en MySQL
two_factor_secret	TEXT / VARCHAR según migración	según PK / admite NULL; confirmar en MySQL
two_factor_recovery_codes	TEXT / VARCHAR según migración	según PK / admite NULL; confirmar en MySQL
two_factor_confirmed_at	datetime	según PK / admite NULL; confirmar en MySQL
deleted_at	datetime	según PK / admite NULL; confirmar en MySQL

3.9. Casos de uso por rol

Rol / módulo	Rutas representativas y controles
Público	/, /servicios, /mapa, /sitios, /actividades y /festividades.
Registro personalizado	POST /registro; /registro/confirmar/{token}; /registro/estado.
Turista	/turista/checkout y /turista/mis-reservas; auth y verified en el grupo; checkout verifica el rol turista.
Administrador	/admin/panel; /admin/emprendimientos/gestion; /admin/festividades/gestion; /admin/reportes. Role: superAdministrador o administrador_gad.
Publicaciones administrativas	/admin/admin/sitios y /admin/admin/actividades; prefijo duplicado existente en esta versión.
Emprendimiento	/emprendimiento/panel; /emprendimiento/mis-servicios/servicios; /emprendimiento/reservas; /emprendimiento/reportes. Role: emprendimiento y middleware emprendimiento.activo.
Configuración	/settings/profile, /settings/password, /settings/appearance, /settings/two-factor.

Rol / módulo	Rutas representativas y controles
Configuración comercial	/settings/emprendimiento y /settings/emprendimiento/datos-pago; rol emprendimiento.
Salud	/up, registrada por bootstrap/app.php.

VerificarEmprendimientoActivo comprueba la asociación del usuario con un establecimiento y su estado. Si está inactivo, cierra la sesión y redirige al acceso. ReservaService limita consultas del turista por user_id y del emprendimiento mediante la relación de servicios; conserva estos filtros al extender funciones.

Fortify proporciona recuperación de contraseña, verificación y 2FA. Los límites configurados son cinco intentos por minuto para acceso y desafío 2FA. El acceso examina el rol; una cuenta sin rol puede ser rechazada. El registro personalizado y el registro estándar de Fortify coexisten como superficies distintas y deben probarse por separado.

Actor	Casos de uso
Turista	Registrarse, verificar correo, consultar catálogo/publicaciones, seleccionar servicios, solicitar reserva, consultar historial, subir comprobante y cancelar cuando esté permitido.
Administrador	Registrar y mantener emprendimientos; administrar sitios, actividades y festividades; consultar reportes.
Emprendedor	Configurar categorías, gestionar servicios y galerías, confirmar/reagendar/cancelar reservas, revisar comprobantes, mantener datos comerciales/bancarios y consultar reportes.

CAPTURA 22 POR INSERTAR — Diagrama de casos de uso
Insertar actores Turista, Administrador y Emprendedor con sus casos de uso dentro del límite del sistema. Diferenciar autenticación y permisos; no inventar módulos que no existen.
Pie sugerido: Figura 22. Diagrama de casos de uso.

3.10. Diagramas de secuencia

Secuencia de reserva: el turista consulta disponibilidad; Livewire valida; ReservaService inicia una transacción, bloquea el servicio conforme al motor, recalcula importes y disponibilidad, escribe reservas y reserva_detalles y confirma. La notificación se emite según el flujo. La secuencia de comprobante valida propiedad/estado, almacena archivo y cambia la reserva a Pago en revisión.

CAPTURA 23 POR INSERTAR — Secuencia de registro y confirmación de correo
Mostrar turista → formulario → RegistroController → validación/caché → Mail → enlace de confirmación → User/rol. Reflejar la caducidad de 30 minutos sin mostrar tokens.
Pie sugerido: Figura 23. Secuencia de registro y confirmación de correo.

CAPTURA 24 POR INSERTAR — Secuencia de solicitud de reserva
Mostrar turista → Checkout → ReservaService → InventarioService/Eloquent → MySQL → resultado y aviso. Incluir transacción, validación de stock y creación de detalles.
Pie sugerido: Figura 24. Secuencia de solicitud de reserva.

CAPTURA 25 POR INSERTAR — Secuencia de comprobante y revisión
Mostrar turista → HistorialReservas → ReservaService → Storage/MySQL; después emprendedor → GestorReservas → cambio de estado. Separar la revisión humana de la tarea programada.
Pie sugerido: Figura 25. Secuencia de comprobante y revisión.

3.11. Interfaz del sistema

Las siguientes capturas deben proceder de la instalación real, preferentemente con datos de demostración. Inclúyelas en las subsecciones correspondientes y conserva los nombres de menú existentes. No publiques cédulas, correos personales, comprobantes o números de cuenta reales.

CAPTURA 26 POR INSERTAR — Interfaz pública e inicio de sesión
Página de inicio y formulario de acceso con los elementos principales.
Pie sugerido: Figura 26. Interfaz pública e inicio de sesión.

CAPTURA 27 POR INSERTAR — Interfaz del turista
Buscador, carrito/checkout e historial de reservas; separar cada pantalla si hace falta.
Pie sugerido: Figura 27. Interfaz del turista.

CAPTURA 28 POR INSERTAR — Interfaz del administrador
Panel y gestión de emprendimientos; como complemento, publicaciones y reportes.
Pie sugerido: Figura 28. Interfaz del administrador.

CAPTURA 29 POR INSERTAR — Interfaz del emprendedor
Gestión de servicios, detalle de reserva, revisión de comprobante y reportes.
Pie sugerido: Figura 29. Interfaz del emprendedor.

CAPTURA 30 POR INSERTAR — Interfaz de configuración de pago
Formulario de cuentas bancarias con datos ficticios y botón para guardar.
Pie sugerido: Figura 30. Interfaz de configuración de pago.

4. Implementación

4.1. Módulos y reglas de negocio

La selección del turista se conserva en sesión mediante reserva_turista_carrito y reserva_turista_datos. Checkout verifica la cuenta y registra la solicitud mediante ReservaService. El flujo del emprendimiento permite buscar a un turista existente por cédula y crear la reserva desde el panel.

Origen / evento	Comportamiento implementado
Reserva del turista	Crea cabecera Pendiente y procesa las líneas del carrito.
Reserva del emprendimiento	Crea cabecera Confirmada.
Confirmación del establecimiento	Cambia la reserva a Confirmada y emite instrucciones por correo.
Comprobante del turista	Solo sobre Confirmada; almacena archivo y fecha; cambia a Pago en revisión.
Revisión del responsable	Puede marcar Completada según las acciones visibles.
Cancelación	Guarda estado y motivo/auditoría según el flujo.
Reagendamiento	Valida fechas y disponibilidad y actualiza los detalles.

InventarioService calcula disponibilidad diaria para las categorías aplicables. Su consulta de ocupación considera estados Confirmada y Pendiente. Pago en revisión y Completada no participan en esa consulta: es una limitación que debe revisarse antes de confiar en el control de cupos en producción.

ReservaService utiliza transacciones y lockForUpdate al procesar servicios. En MySQL, utiliza InnoDB y comprueba índices, aislamiento y comportamiento de bloqueos. No se verificó prevención de sobreventa con concurrencia real en el servidor del usuario.

Categoría	Fórmula del subtotal en ReservaService
Paquete turístico	precio × cantidad.
Hospedaje	precio × personas × cantidad × días.
Guianza / Alquiler	precio × cantidad × días.
Alimentación y otras	precio × cantidad.

El número de días se calcula de forma inclusiva, con mínimo uno. El precio_unitario del detalle conserva el precio al registrar la reserva. El total suma los subtotales calculados. Estas fórmulas describen el código; confirma con negocio si un hospedaje debe cobrarse por días inclusivos o por noches.

La búsqueda pública exige tres días de anticipación. Los paquetes no pueden empezar en domingo ni lunes. La cancelación por turista exige más de 24 horas antes del primer detalle; se permite en Pendiente o Confirmada sin comprobante dentro del plazo previsto.

La tarea de cada minuto cancela Pendientes con 24 horas o más desde la creación; cancela Confirmadas sin comprobante con 24 horas o más desde la actualización; avisa al responsable en la ventana de 23 a 24 horas desde el comprobante y marca Completada al superar 24 horas en revisión. También se invoca procesamiento al abrir la gestión de reservas del emprendimiento.

Existe un aviso de interfaz que menciona 12 horas para aprobación automática, mientras ReservaService usa 24. No comunique un plazo garantizado hasta resolver la discrepancia. Una tarea fallida, un cron detenido o un correo con error pueden retrasar el procesamiento.

Las imágenes y comprobantes se guardan en el disco public bajo storage/app/public y se exponen con public/storage. Una ruta pública no protege por sí misma un comprobante bancario. Evalúa almacenamiento privado y descarga autorizada para documentos sensibles antes de producción.

Integración	Necesidad y comportamiento
Correo	Confirmación de registro, recuperación y avisos de reservas. MAIL_MAILER=log solo simula la entrega en registros.
PDF	Dompdf genera los reportes; el controlador del emprendimiento revisado devuelve PDF.
Mapa	Leaflet desde unpkg.com; capas de cartocdn.com, ArcGIS y OpenTopoMap.
Traducción pública	Script translate.google.com; la dependencia PHP cloud-translate declarada no prueba que exista una integración de servidor activa.
Estilos públicos	cdn.tailwindcss.com; su bloqueo deja algunas vistas sin estilos.
Fuentes	fonts.bunny.net en vistas con head compartido.

Mantén TLS y verificaciones de integridad de los gestores de paquetes. Añade únicamente los destinos necesarios a la política de red. Si el CDN no está disponible, debe corregirse el acceso o adoptarse una solución de recursos locales mediante un cambio de código separado.

4.2. Descripción de funciones y pseudocódigo

Función / componente	Responsabilidad y pasos
RegistroController: store / confirmar	Validar datos → guardar solicitud temporal → enviar correo → comprobar token → crear usuario → asignar turista.
Checkout: confirmar	Comprobar sesión y rol → recuperar carrito → registrar reserva → limpiar sesión → mostrar resultado.
ReservaService: registrarReservaTurista	Iniciar transacción → crear Pendiente → procesar líneas y total → confirmar transacción → emitir aviso.
ReservaService: subirComprobante	Verificar reserva del usuario y estado Confirmada → almacenar imagen → guardar fecha → cambiar a Pago en revisión → notificar.
GestorReservas / cambiarEstado	Comprobar reserva del emprendimiento → validar acción → actualizar estado/auditoría → comunicar resultado.
InventarioService	Consultar ocupación del rango según categoría/estados → calcular ocupación máxima → restar del stock → devolver disponibilidad.
procesarCancelacionesAutomaticas	Consultar vencimientos → cancelar pendientes/confirmadas sin pago → avisar sobre revisión → completar según regla existente.

Los pasos describen el código inspeccionado. No sustituyen la validación de comportamiento en MySQL; revisa consultas y bloqueos bajo concurrencia antes de afirmar prevención de sobreventa.

CAPTURA 31 POR INSERTAR — Código de las funciones principales
Mostrar fragmentos de ReservaService e InventarioService con firmas y reglas relevantes. Omitir configuración privada y no sustituir el código por ejemplos no implementados.
Pie sugerido: Figura 31. Código de las funciones principales.

4.3. Despliegue y actualización

Procedimiento recomendado: respaldar; preparar la versión en pruebas; instalar dependencias desde lockfiles; compilar assets; configurar el entorno; aplicar migraciones revisadas; publicar con raíz public; iniciar PHP-FPM, colas y programador; validar comportamiento. No se ejecutó este procedimiento en producción durante la elaboración.

```
composer install --no-dev --no-interaction --prefer-dist --optimize-autoloader
npm ci
npm run build
php artisan migrate --force
php artisan storage:link
php artisan config:cache
php artisan view:cache
php artisan queue:restart
php artisan schedule:list
```

Ejecuta estos comandos solo en el despliegue preparado y con autorización operativa. migrate --force aplica cambios reales. No regeneres APP_KEY. No ejecutes migrate:fresh en datos reales. Verifica el comportamiento después de cachear configuración; limpia cachés cuando cambies variables, rutas o vistas.

Comprueba /up, acceso, registro personalizado, recuperación, permisos, catálogo, reserva, comprobante y reporte. Una respuesta de salud no demuestra que correo, inventario o pagos funcionen. Supervisa la URL de APP_URL, HTTPS, sesiones, tamaño de carga y permisos de los directorios.

CAPTURA 32 POR INSERTAR — Despliegue y acceso operativo
Mostrar la ejecución satisfactoria de migraciones y compilación, y una comprobación de acceso de la versión desplegada. Distinguir entorno de pruebas de producción.
Pie sugerido: Figura 32. Despliegue y acceso operativo.

4.4. Pruebas y simulación

CI prepara PHP 8.4/8.5 y Node 22, instala dependencias, genera una clave de prueba, compila assets y ejecuta Pest. El script composer test incluye primero lint:check; si el formato falla, las pruebas pueden no llegar a ejecutarse. Para diagnosticar por separado, ejecuta comprobación de formato y pruebas directamente.

```
composer check-platform-reqs
php vendor/bin/pint --test
php artisan test
# Solo con una base aislada y datos de prueba:
php artisan simulacion:rendimiento --concurrency=3 --iterations=10 --mode=mixed
```

El simulador admite modos create, update y mixed; limita concurrencia a 100 e iteraciones a 1000 y escribe reportes en storage/simulacion_reportes. Sus umbrales están en SimulacionConfig. El nombre ISO 25023 en el comando no constituye una certificación; la simulación no sustituye una prueba HTTP de carga representativa.

Verificación local del 12 de octubre de 2026: composer check-platform-reqs pasó con PHP 8.4.26; las 22 migraciones figuraron aplicadas; route:list y schedule:list se ejecutaron. php artisan test ejecutó 52 pruebas: 43 aprobadas y 9 fallidas, con 142 aserciones. La suite no está aprobada por completo. Esta ejecución anterior utilizó una base local de prueba distinta de MySQL; NO valida el servidor MySQL del usuario y no debe presentarse como evidencia de esa instalación.

Grupo de fallos	Observación del resultado
Acceso y desafío 2FA	Usuarios de prueba sin rol son rechazados por la autenticación.
Recuperación	Tres pruebas esperan la notificación estándar ResetPassword que no fue observada.
Registro estándar	Binding de CreatesNewUsers no resoluble en ese flujo de prueba.
Dashboard	Prueba espera 200, pero el comportamiento observado devuelve redirección 302.
Configuración 2FA	Prueba busca texto inglés que no aparece en la respuesta.

Estos resultados corresponden a Rama-Daniela y reemplazan cualquier resultado antiguo de pruebas de main como evidencia para este manual. No se modificaron fuentes o tests para hacerlos pasar. No se probaron pagos reales, entrega SMTP, motores remotos, carga concurrente ni restauración de producción.

CAPTURA 33 POR INSERTAR — Pruebas contra MySQL
Mostrar comandos, base de PRUEBAS y resultado actual. phpunit.xml puede usar una conexión de prueba distinta: configurar y verificar una base MySQL dedicada antes de ejecutar; RefreshDatabase puede eliminar/recrear tablas. Nunca apuntar las pruebas a producción.
Pie sugerido: Figura 33. Pruebas contra MySQL.

4.5. Limitaciones y diagnóstico

Situación	Diagnóstico / acción recomendada
Vista de sitio turístico falla	LandingController referencia turista.publicacion.detalle-sitio, pero el archivo inspeccionado está bajo livewire/turista/publicacion. Revisar la referencia.
Registro de usuarios administrativo	Ruta existente, menú no la muestra y formulario omite campos exigidos; no presentarlo como flujo completo.
Plazo del pago inconsistente	Aviso 12 h frente a procesamiento 24 h; unificar reglas y mensajes.
Disponibilidad tras cambio de estado	La ocupación considera solo Pendiente y Confirmada; evaluar revisión de pagos y completadas.
Recaudación	El indicador incluye Confirmadas y Completadas; no equivale necesariamente a cobros comprobados.
SMTP o cron falla	Revisar logs, configuración efectiva y proceso programado; no reenviar masivamente sin comprobar efectos.
Clases ausentes / extensiones	Validar autoload y check-platform-reqs; instalar requisitos sin ignorarlos.

Situación	Diagnóstico / acción recomendada
CSS público ausente	Comprobar CDN Tailwind; el build de Vite por sí solo no resuelve esa dependencia pública.

Estas observaciones requieren trabajo de código u operación separado. La generación y publicación del manual no las corrige. Al reportar un incidente, aporta el commit, ruta, rol y pasos, con registros sanitizados sin credenciales ni datos personales.

Anexo. Control de cambios y guía de capturas

Edición 2.0: motor operativo corregido a MySQL; conexión, extensión PHP, creación de esquema, respaldos/restauración y concurrencia adaptados; estructura alineada al Manual Técnico CAE.pdf; agregadas ubicaciones y descripciones de capturas. Se mantienen las reglas y limitaciones verificadas de Rama-Daniela. No se certifica una instalación MySQL no inspeccionada.

El PDF CAE se utiliza como referencia de organización, no como fuente de tecnologías, autores, procesos económicos o funciones para Explora Candelaria. No se modificaron aplicación, configuración de conexión real ni datos del usuario.